



Unidad 2.

Administración y Optimización Índices en BD. Relacionales

INFO 209 – Taller de Bases de Datos
Juan Pablo Salazar, 2026

O'REILLY®

Efficient MySQL Performance

Best Practices and Techniques



Daniel Nichter

2. Indexes and Indexing..... 35

Red Herrings of Performance 37

Better, Faster Hardware! 37

MySQL Tuning 39

MySQL Indexes: A Visual Introduction 40

InnoDB Tables Are Indexes 41

Table Access Methods 45

Leftmost Prefix Requirement 49

EXPLAIN: Query Execution Plan 51

WHERE 54

GROUP BY 60

ORDER BY 65

Covering Indexes 71

Join Tables 71

Indexing: How to Think Like MySQL 80

Know the Query 80

Understand with EXPLAIN 81

Optimize the Query 82

Deploy and Verify 83

It Was a Good Index Until... 84

Queries Changed 84

Excessive, Duplicate, and Unused 85

Extreme Selectivity 86

It's a Trap! (When MySQL Chooses Another Index) 87

Table Join Algorithms 87

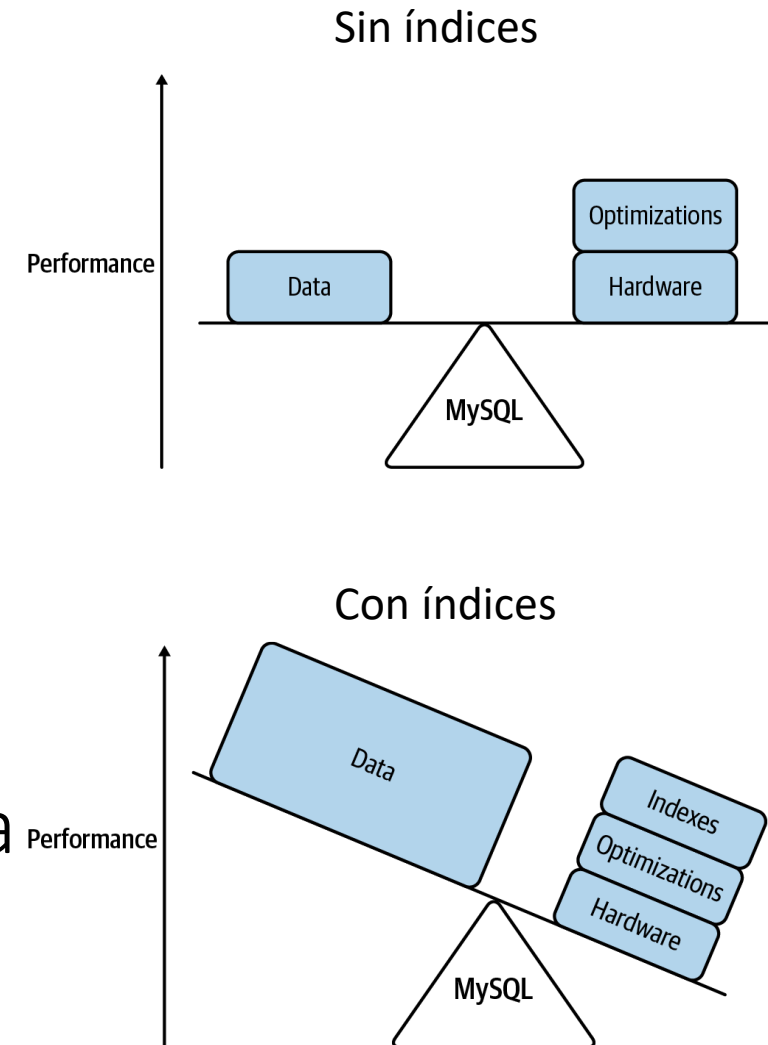
Summary 89

Practice: Find Duplicate Indexes 90

Estrategias para mejorar performance

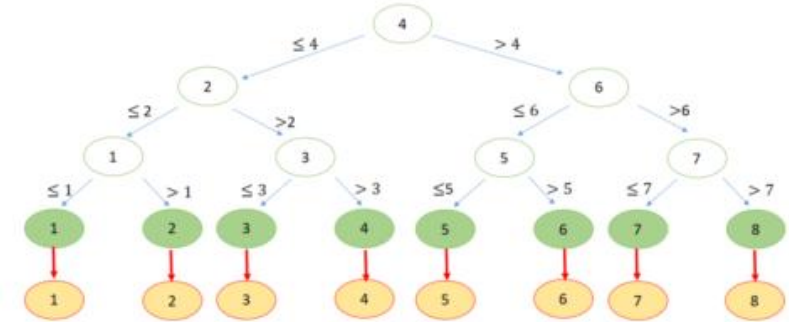
- Tuning (ajustar variables internas, cercano a R&D).
- Configuring (ajustar parámetros).
- **Optimizing** (reducir carga de trabajo en MySQL).

Escalar en hardware **nunca** es la primera opción.



MySQL utiliza índices B-Tree.

- Los árboles **no necesariamente** son binarios.
- Por simplicidad, **asumiremos** que son binarios.



En MySQL, **son estructuras separadas** de las tablas.

(Salvo la llave primaria, que es parte de la tabla).

Operaciones básicas.

Búsqueda en una tabla

Sin índice

Orden (N)

Con llave primaria

Orden ($\log_2 N$)

Join de 2 tablas

Sin índice

Orden (N^2)

Con índice en una de ellas

Orden ($N \log_2 N$)



¿Cómo creo un índice?

UACH

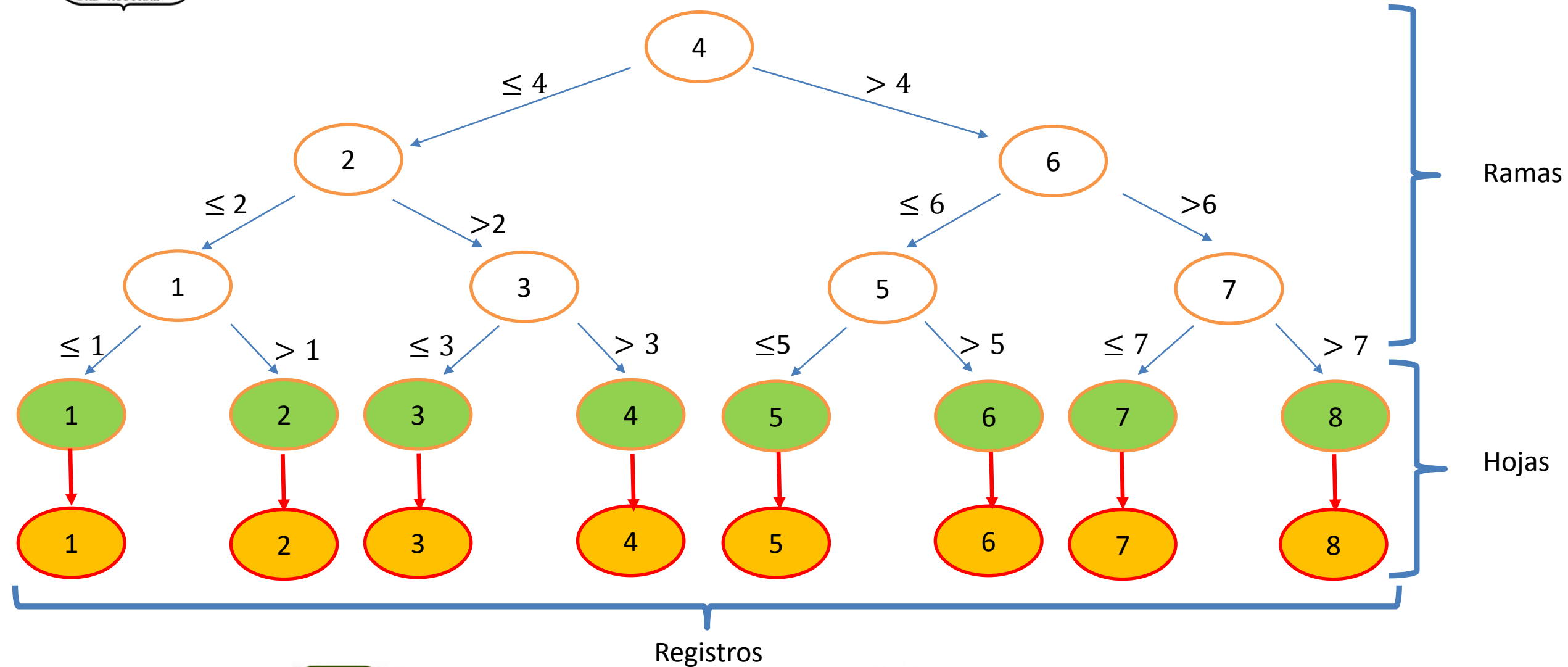
- Llave primaria (agiliza validación de unicidad).
- Llave foránea (agiliza validaciones de foreign keys).
- Índice diferente. Ejm:

```
create index indice on sakila.rental (return_date);
```



B-Tree – Primary Key

UACH





Cálculo de alturas de árbol

UACH

Si tengo 2 registros

Si tengo 16 registros.

Si tengo 30 registros.

Si tengo 32 registros.

Si tengo 35 registros.



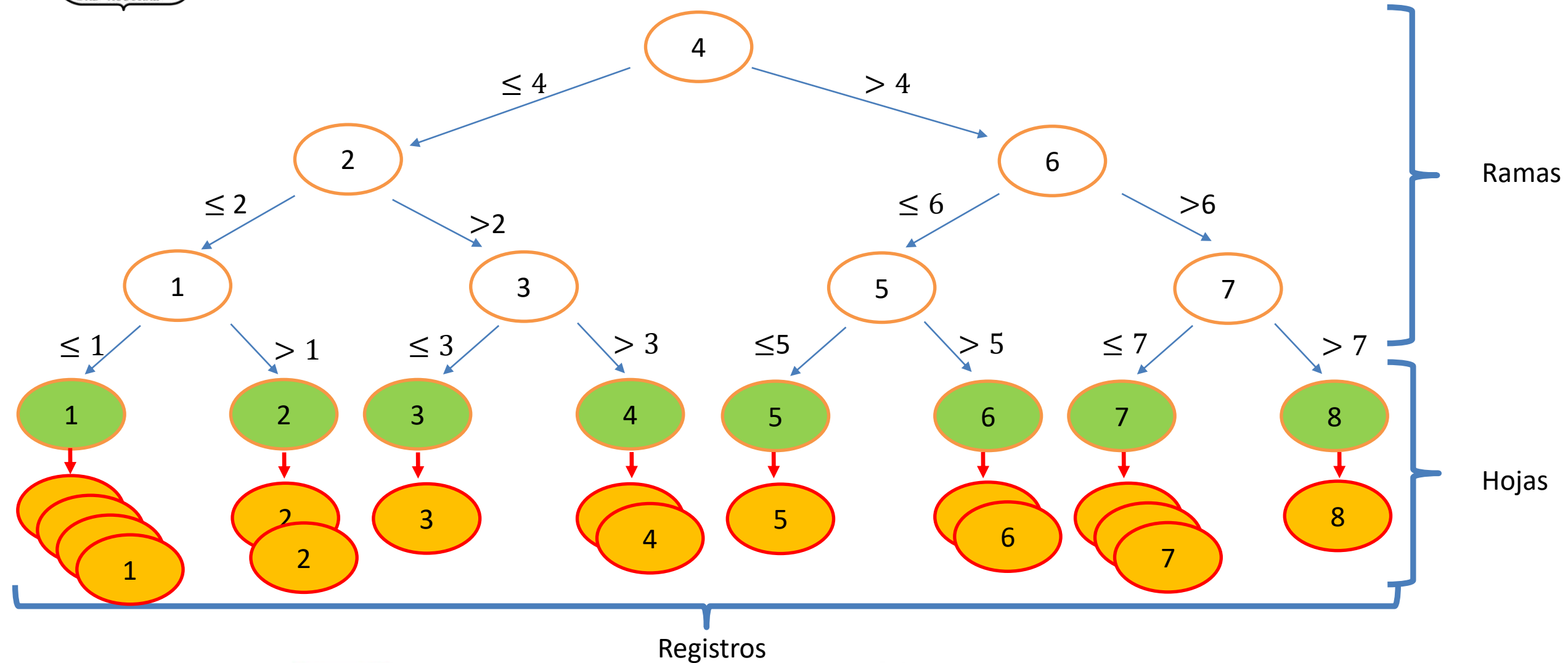
```

graph TD
    4((4)) -- "≤ 4" --> 2((2))
    4 -- "> 4" --> 6((6))
    2 -- "≤ 2" --> 1((1))
    2 -- "> 2" --> 3((3))
    6 -- "≤ 6" --> 5((5))
    6 -- "> 6" --> 7((7))
    1 -- "≤ 1" --> 1_1((1))
    1 -- "> 1" --> 2_1((2))
    3 -- "≤ 3" --> 3_1((3))
    3 -- "> 3" --> 4_1((4))
    5 -- "≤ 5" --> 5_1((5))
    5 -- "> 5" --> 6_1((6))
    7 -- "≤ 7" --> 7_1((7))
    7 -- "> 7" --> 8_1((8))
    style 4 fill:#d9ead3,stroke:#333,stroke-width:1px
    style 2 fill:#d9ead3,stroke:#333,stroke-width:1px
    style 6 fill:#d9ead3,stroke:#333,stroke-width:1px
    style 1 fill:#d9ead3,stroke:#333,stroke-width:1px
    style 3 fill:#d9ead3,stroke:#333,stroke-width:1px
    style 5 fill:#d9ead3,stroke:#333,stroke-width:1px
    style 7 fill:#d9ead3,stroke:#333,stroke-width:1px
    style 1_1 fill:#55a868,stroke:#333,stroke-width:1px
    style 2_1 fill:#55a868,stroke:#333,stroke-width:1px
    style 3_1 fill:#55a868,stroke:#333,stroke-width:1px
    style 4_1 fill:#55a868,stroke:#333,stroke-width:1px
    style 5_1 fill:#55a868,stroke:#333,stroke-width:1px
    style 6_1 fill:#55a868,stroke:#333,stroke-width:1px
    style 7_1 fill:#55a868,stroke:#333,stroke-width:1px
    style 8_1 fill:#55a868,stroke:#333,stroke-width:1px
  
```

Hojas distintas = N

(*) Siempre usaremos el entero superior.

#Operaciones de lectura = $\log_2(N) + 1$



Ejm: Select * from Tabla where atributo = 1;

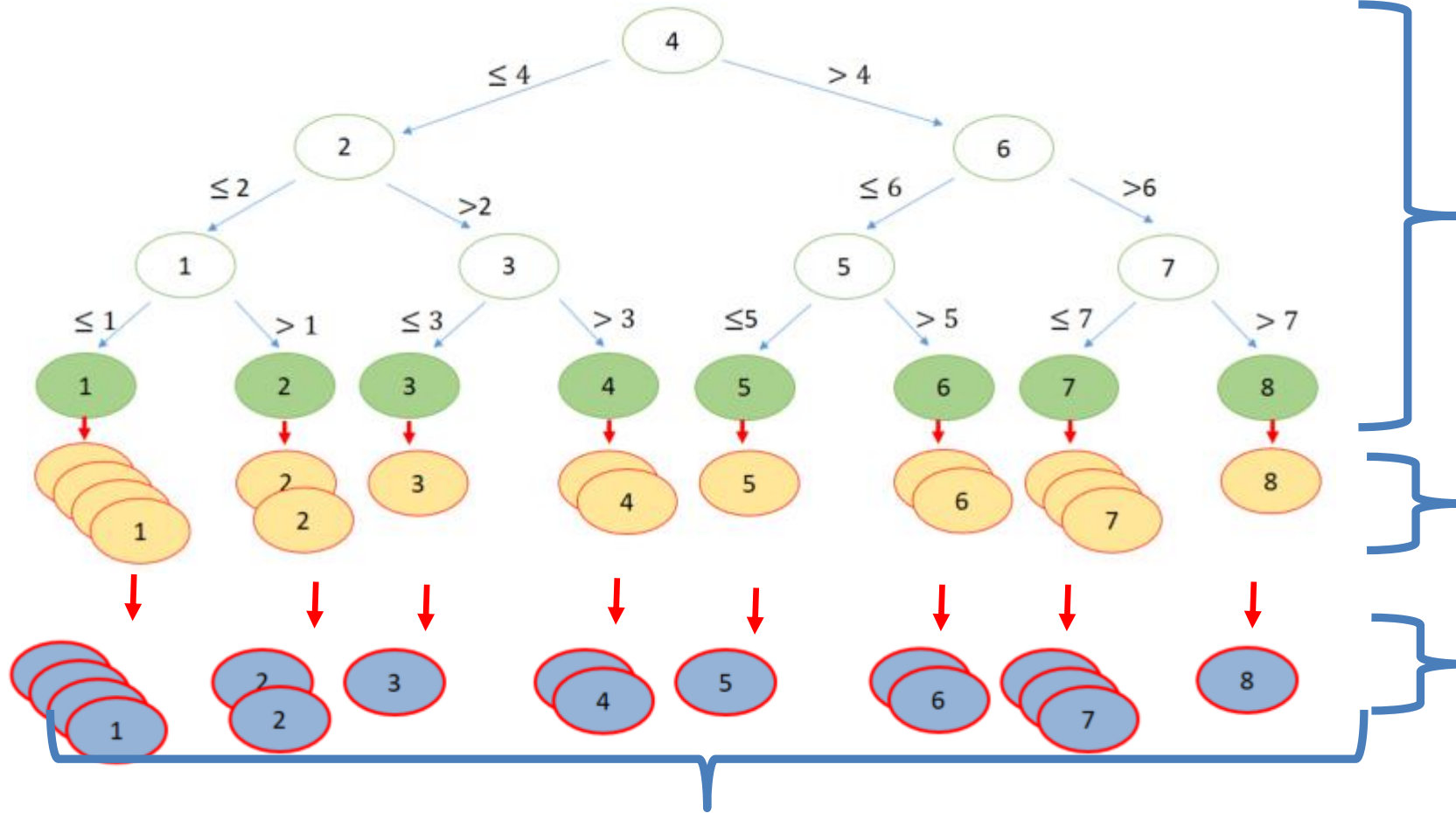


Tabla con N registros distintos

Altura del árbol = $\log_2(M) + 1$

Claves distintas = M

Punteros a llaves primarias

Promedio registros leídos = $\frac{N}{M}$

Registros de datos
(búsqueda de llave primaria en cada uno)

Promedio registros leídos =

$$\frac{N}{M} * (\log_2(N) + 1)$$

$$\text{\#Operaciones de lectura} = \log_2(M) + 1 + \frac{N}{M} * (1 + \log_2(N) + 1)$$



Veamos algunos ejemplos (llave primaria)

UACH

Tabla T, 1000 registros. Atributo pk es su **llave primaria**

Select * from T where pk=1 /* retorna 1 registro!! */

¿Cuántas operaciones de lectura se realizan?

N = 1000

#Operaciones = $\log_2(N) + 1$ (recorrer árbol + hoja + puntero + registro)

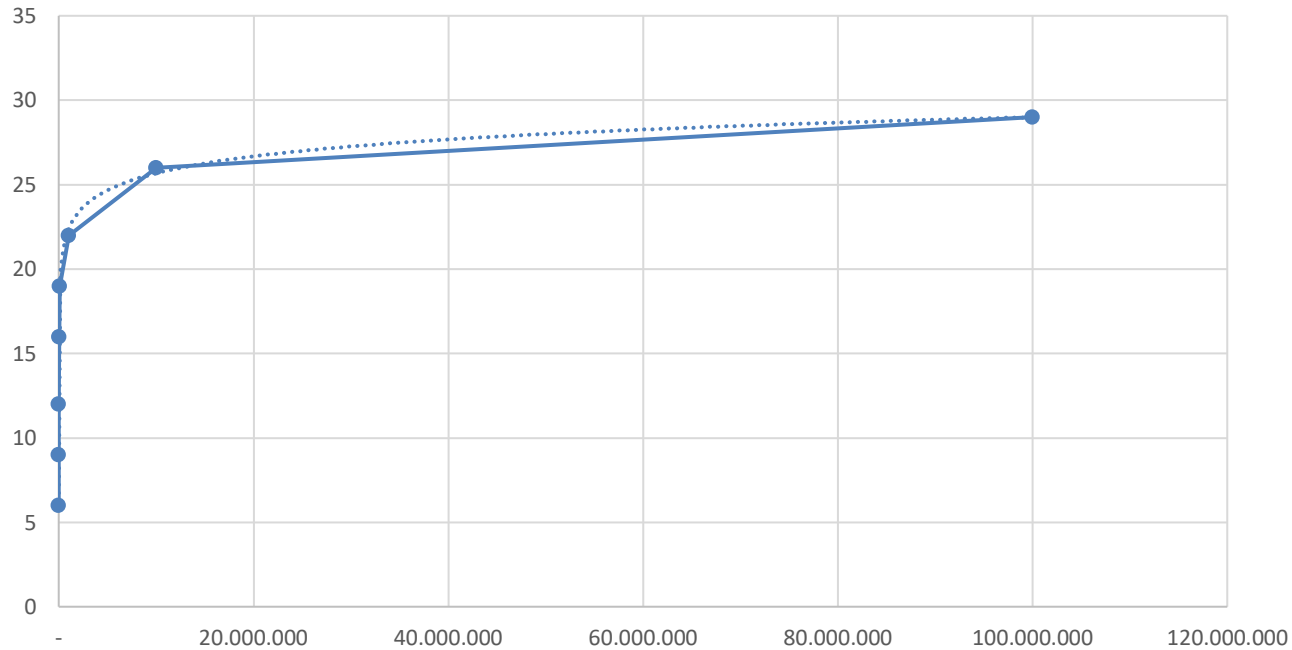
#Operaciones = 10+1=11



Veamos un gráfico (PK & Access Full)

UACH

N° de registros & N° Operaciones (PK)



N° Registros	N° Operaciones (PK)
10	5
100	8
1,000	11
10,000	15
100,000	18
1,000,000	21
10,000,000	25
100,000,000	28

Muy buena escalabilidad (Orden ($\log_2 N$))



Veamos algunos ejemplos (acceso secuencial)

UACH

Tabla T, 1000 registros. Atributo a **no tiene índice**

Select * from T where a=1 /* retorna 5 registros!! */

¿Cuántas operaciones de lectura se realizan?

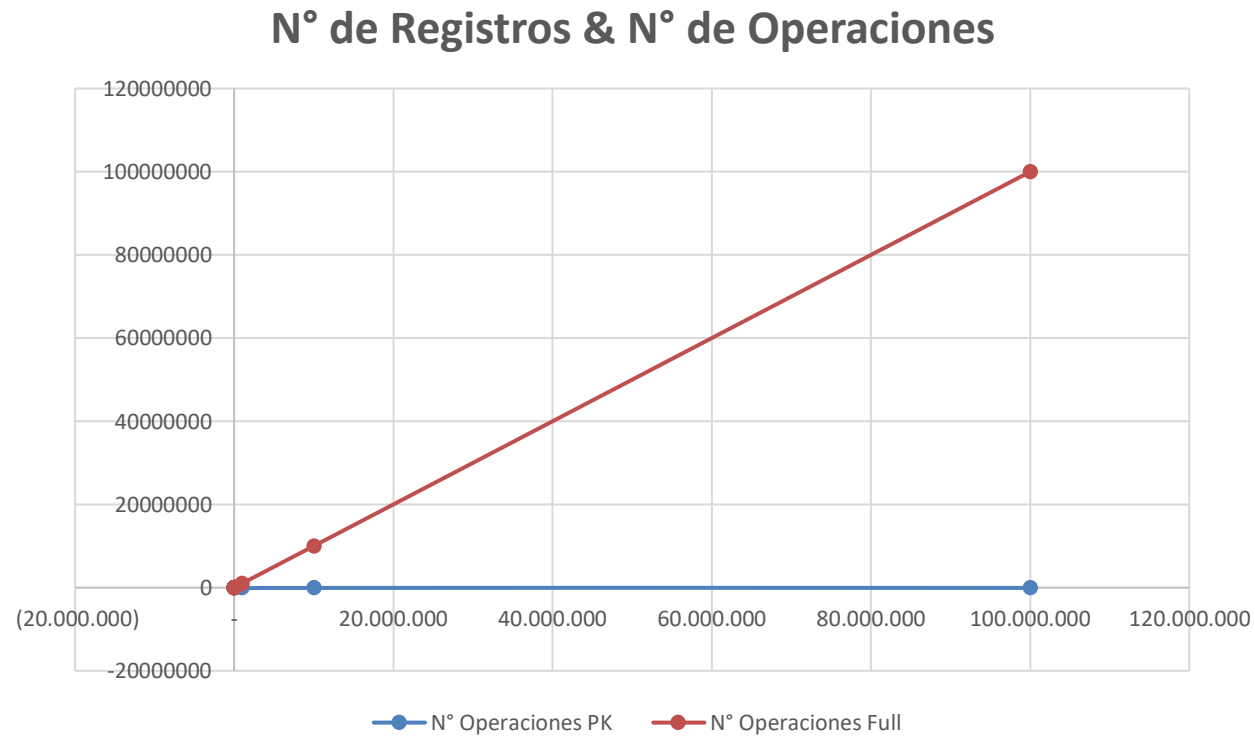
#Operaciones = N

#Operaciones = 1000



Veamos un gráfico (PK & Access Full)

UACH



Full tiene escalabilidad lineal ($\text{Orden}(N)$)



Veamos algunos ejemplos (índice no único)

UACH

Tabla T2, 1000 registros. Atributo a tiene **índice no único**

Select * from T where a=1 /* retorna 5 registros */

¿Cuántas operaciones de lectura se realizan?

N = 1000; M=200

#Operaciones = $\log_2(M) + 1 + \frac{N}{M}$ (hoja + búsqueda por llave primaria)

#Operaciones = $8 + 1 + 5 * (1 + \log_2(1000) + 1) = 8 + 1 + 5 * 12 = 69$

Comparado con acceso por llave primaria (11), y con Full Scan (1000), es bastante bueno.



Veamos algunos ejemplos (índice no único)

UACH

Tabla T2, 1.000.000 registros. Atributo sexo tiene **índice no único**

Select * from T2 where sexo='F' /* retorna 500.000 registros!! */

¿Cuántas operaciones de lectura se realizan?

$N = 1.000.000$; $M=2$

#Operaciones = $\log_2(M) + 1 + \frac{N}{M} (1 + \log_2(1.000.000) + 1)$

#Operaciones = $2+1 + 500.000 * (1+20+1) \approx 10.000.000$!!

Conclusión: El índice por sexo **no sirve** de nada.



- Acceso sin índices es secuencial.
- Acceso por llave primaria es muy eficiente.
- Ganancia en acceso utilizando índices no únicos, depende de las cardinalidades.
 - Índice muy selectivo, rendimiento cercano a llave primaria.
 - Índice poco selectivo, rendimiento cercano a acceso secuencial.

¿Cómo saber de qué manera se utilizan los índices?



Explain Plan

UACH

Nos entrega información relativa a cómo **MySQL** ejecuta las instrucciones y al esfuerzo (costo) necesario.

- Select
- Insert
- Update
- Delete

Requiere los privilegios necesarios para ejecutar instrucciones.

Nos ayuda a **evaluar** efecto de agregar/quitar índices.

Funciona mejor si el sistema tiene **estadísticas** de tablas e índices.

```
Ejm: analyze table sakila.film
```



```
select * from information_schema.statistics  
where table_name = 'FILM';
```

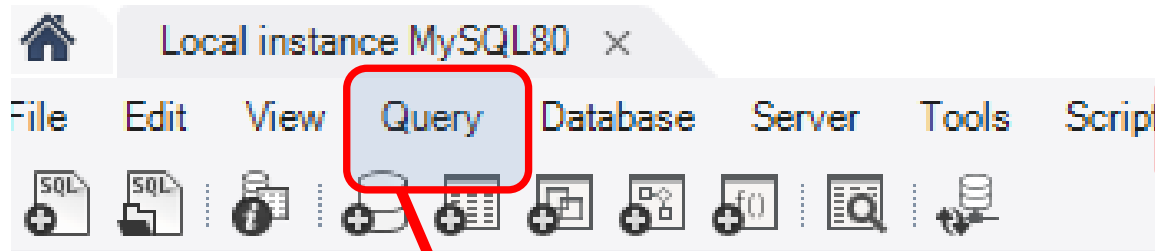
¿Qué información entrega?

- Cardinalidad de índices
- Tipo de cada índice (B-Tree, Hash)
- Si el índice es único o no.

Ejm:

- El índice por **language** tiene un único valor.
- El índice por **film_id** tiene 1000 valores distintos.

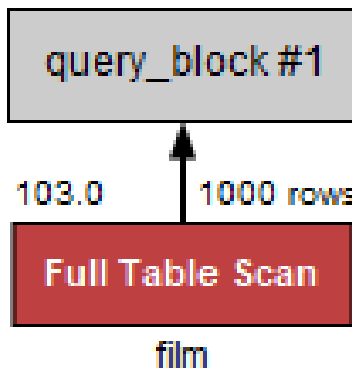
Explain ¿Cómo se utiliza?



Opción: Explain Current Statement

```
select * from sakila.film;
```

Query cost: 103.00



film

Access Type: ALL
Full Table Scan
Cost Hint: Very High - very costly for large tables (not so much for small ones).
No usable indexes were found for the table and the optimizer must search every row.
This could also mean the search range is so broad that the index would be useless.

Used Columns: film_id,
title,
description,
release_year,
language_id,
original_language_id,
rental_duration,
rental_rate,
length,
replacement_cost,
rating,
special_features,
last_update

Key/Index: -

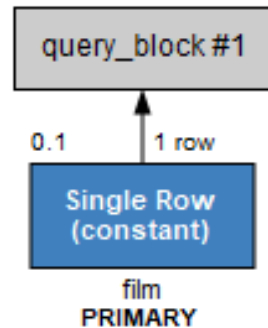
Rows Examined per Scan: 1000
Rows Produced per Join: 1000
Filtered (ratio of rows produced per rows examined): 100.00%
Hint: 100% is best, <= 1% is worst
A low value means the query examines a lot of rows that are not returned.

Cost Info
Read: 3.00
Eval: 100.00
Prefix: 103.00
Data Read: 539K

Explain – Consulta que utiliza llave primaria

```
select * from sakila.film  
where film_id = 1;
```

Query cost: 1.00



film

Access Type: const
Single Row
(constant)
Cost Hint: Very low cost

Used Columns: film_id,
title,
description,
release_year,
language_id,
original_language_id,
rental_duration,
rental_rate,
length,
replacement_cost,
rating,
special_features,
last_update

Key/Index: PRIMARY
Ref.: const
Used Key Parts: film_id
Possible Keys: PRIMARY

Rows Examined per Scan: 1
Rows Produced per Join: 1
Filtered (ratio of rows produced per rows examined): 100.00%
Hint: 100% is best, <= 1% is worst
A low value means the query examines a lot of rows that are not returned.

Cost Info

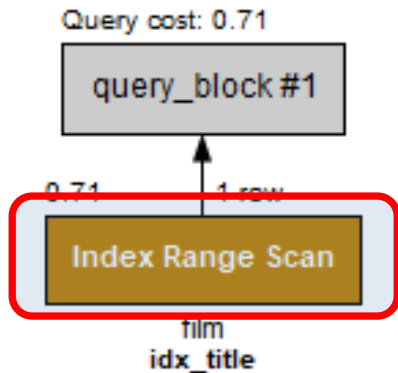
Read: 0.00
Eval: 0.10
Prefix: 0.00
Data Read: 552



Explain – Consulta que utiliza índice

UACH

```
SELECT * FROM sakila.film  
where title like 'DINOSAUR%'
```



film

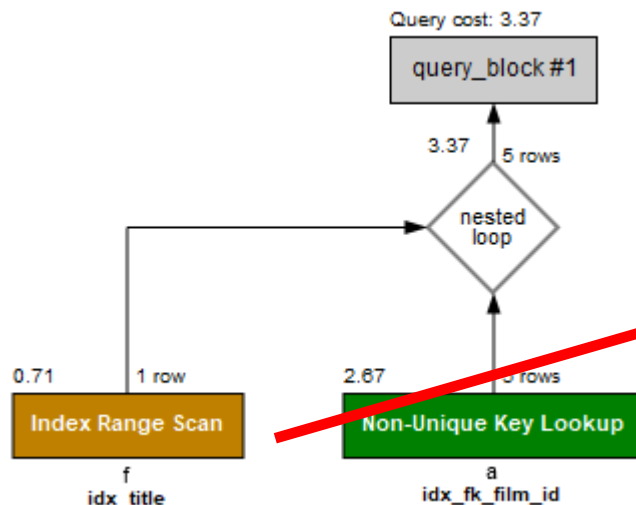
Access Type: range
Index Range Scan
Cost Hint: Medium - partial index scan
Used Columns: film_id,
title,
description,
release_year,
language_id,
original_language_id,
rental_duration,
rental_rate,
length,
replacement_cost,
rating,
special_features,
last_update

Key/Index: idx_title
Used Key Parts: title
Possible Keys: idx_title

Rows Examined per Scan: 1
Rows Produced per Join: 1
Filtered (ratio of rows produced per rows examined): 100.00%
Hint: 100% is best, <= 1% is worst
A low value means the query examines a lot of rows that are not returned.

Cost Info
Read: 0.61
Eval: 0.10
Prefix: 0.71
Data Read: 552

SELECT * FROM sakila.film f, sakila.film_actor a
where f.film_id= a.film_id and
title like 'DINOSAUR%'



Access Type: range
Index Range Scan
Cost Hint: Medium - partial index scan

Used Columns: film_id,
title,
description,
release_year,
language_id,
original_language_id,
rental_duration,
rental_rate,
length,
replacement_cost,
rating,
special_features,
last_update

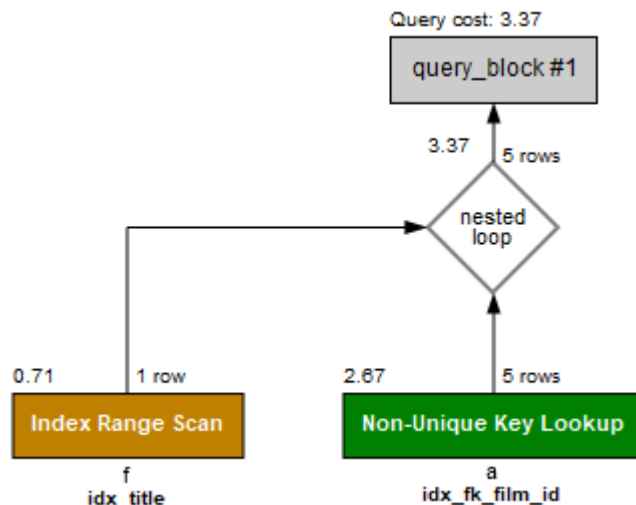
Key/Index: idx_title
Used Key Parts: title
Possible Keys: PRIMARY,
idx_title

Rows Examined per Scan: 1
Rows Produced per Join: 1
Filtered (ratio of rows produced per rows examined): 100.00%

Hint: 100% is best, <= 1% is worst
A low value means the query examines a lot of rows that are not returned.

Costs:
Read: 0.61
Eval: 0.10
Prefix: 0.71
Data Read: 552

SELECT * FROM sakila.film f, sakila.film_actor a
where f.film_id= a.film_id and
title like 'DINOSAUR%'



Access Type: ref
Non-Unique Key Lookup
Cost Hint: Low-medium - Low if number of matching rows is small, higher as the number of rows increases.
Used Columns: actor_id,
film_id,
last_update

Key/Index: idx_fk_film_id
Ref.: sakila.f.film_id
Used Key Parts: film_id
Possible Keys: idx_fk_film_id

Rows Examined per Scan: 5
Rows Produced per Join: 5
Filtered (ratio of rows produced per rows examined): 100.00%
Hint: 100% is best, <= 1% is worst
A low value means the query examines a lot of rows that are not returned.

Cost Info
Read: 2.12
Eval: 0.55
Prefix: 3.37
Data Read: 87

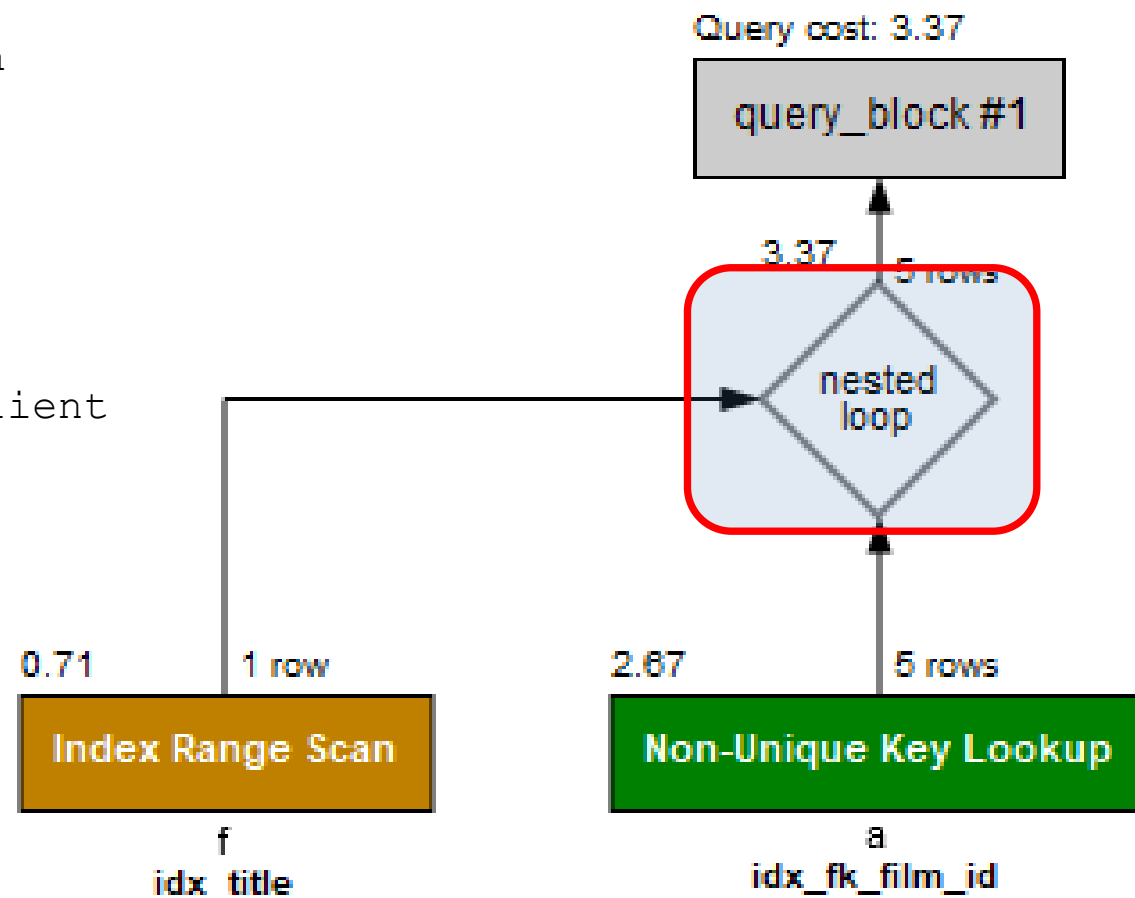


Explain – Consulta que accede a 2 tablas

UACH

```
SELECT * FROM sakila.film f, sakila.film_actor a
WHERE
  f.film_id= a.film_id and
  title like 'DINOSAUR%'
```

```
For each row in f matching range {
  for each row in a matching reference key {
    if row satisfies join condition, send to client
  }
}
```

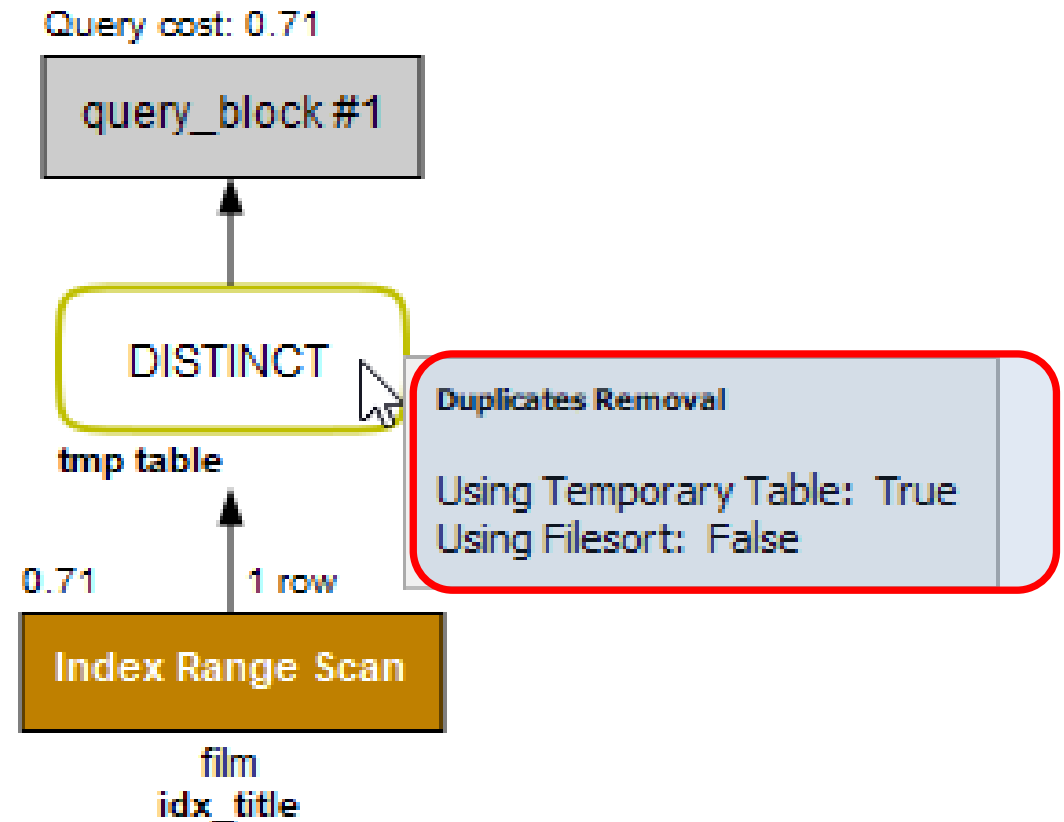




Explain – Consulta que utiliza tabla temporal

UACH

```
select distinct replacement_cost  
from sakila.film  
where title like 'DINOSAUR%'
```



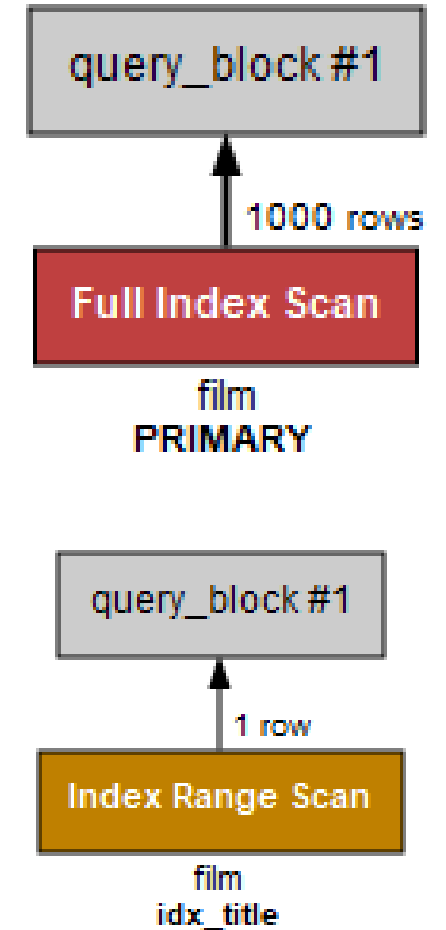


Explain – Update

UACH

```
update sakila.film  
set replacement_cost= replacement_cost * 1.1;
```

```
update sakila.film set replacement_cost= replacement_cost * 1.1  
where title like 'DINOSAUR%';
```





Vamos ahora a MySQL Workbench

UACH

The screenshot shows the MySQL Workbench interface for a local instance of MySQL 8.0. The left sidebar contains a 'Navigator' pane with 'SCHEMAS' and 'Administration' tabs. Under 'SCHEMAS', the 'film' table is selected. The 'Information' pane shows the table's columns and their data types.

The main editor displays a SQL query:

```
4 • select * from sakila.film;
5
6 • select c.first_name, c.last_name, count(*)
7   from sakila.customer c join sakila.rental r
8   on c.customer_id = r.customer_id
```

The 'Visual Explain' tab is active, showing the execution plan for the second query. The plan indicates a 'Full Table Scan' on the 'film' table, resulting in 1000 rows and a cost of 103.0. The overall query cost is 103.00.

The bottom of the interface shows the 'Output' pane and buttons for 'Apply', 'Revert', 'Context Help', and 'Snippets'.